Outil d'analyse de langages

Hategekimana Fabrice

09 mars 2021

Motivation:

Ce que nous voulons:

- Étude de langage
- Sémantique opérationnelle

Créer un outil pour:

- Définir la sémantique d'un langage
- En voir ses dérivations

Outil

langage:

- Prolog like langage
- Syntaxe simplifiée pour la définition de langage
- Utilisation de PLY en tant que lexer/parser

interface

- CLI -> GUI
- visualisation par graphe

Structure du langage

En construction. . .

Sémantique sur les types

Signature

$$\Sigma =< S, F >$$

Depuis sigma on peut dériver l'ensemble T_{sigma} qui est l'ensemble des termes généré par Sigma.

Syntaxe

Typage

```
true[empty,bool].
not[bool,bool].
and[bool,bool,bool].
```

Traduction

```
$ true => ['bool']
$ false => ['bool']
$ and(_,_) => ['bool']
```

Programme

```
P= { inst1;;inst2;;...;;instn }
```

Strucutre du domaine Sémantique

Sémantique sur les valeurs de retour

```
- arithmétique
- langage plus complexe
```

Etat du système

```
State(e1,e2,...,en)
```

Transition

```
Transition(State(...),inst) => State(...)
```

Syntaxe côté utilisateur

State

```
<e1,...,en>
```

State x inst

```
<e1,...,en,inst>
```

Transition

```
<e1,...,en,inst> => <i1,...,in>
```

Définition linéaire

```
e1 => i1,...,in => in -- <e1,...,en,inst> => <i1,...,in>
```

Définition implicite

$$\textsc{Empty} : \quad \frac{}{\langle Tab, Pt, \epsilon \rangle \Rightarrow_P \langle Tab, Pt \rangle}$$

$$\textsc{Prog} : \quad \frac{\langle Tab, Pt, I \rangle \Rightarrow_I \langle Tab', Pt' \rangle, \langle Tab', Pt', P \rangle \Rightarrow_P \langle Tab'', Pt'' \rangle}{\langle Tab, Pt, I\ P \rangle \Rightarrow_P \langle Tab'', Pt'' \rangle}$$

Figure 2: definition_implicite

Limitations(1)

Structure de données:

```
node(leaf,node(node(leaf,leaf),leaf)) => tree
```

Liste

```
[] => list
list(a,list(b,list(c,d)) => list
[a,b,c,d] => list
```

Limitations(2)

Exercice 5

$$\text{Cas } + : \quad \frac{modify(Tab, Pt, 1) = Tab'}{\langle Tab, Pt, + \rangle \Rightarrow_I \langle Tab', Pt \rangle}$$

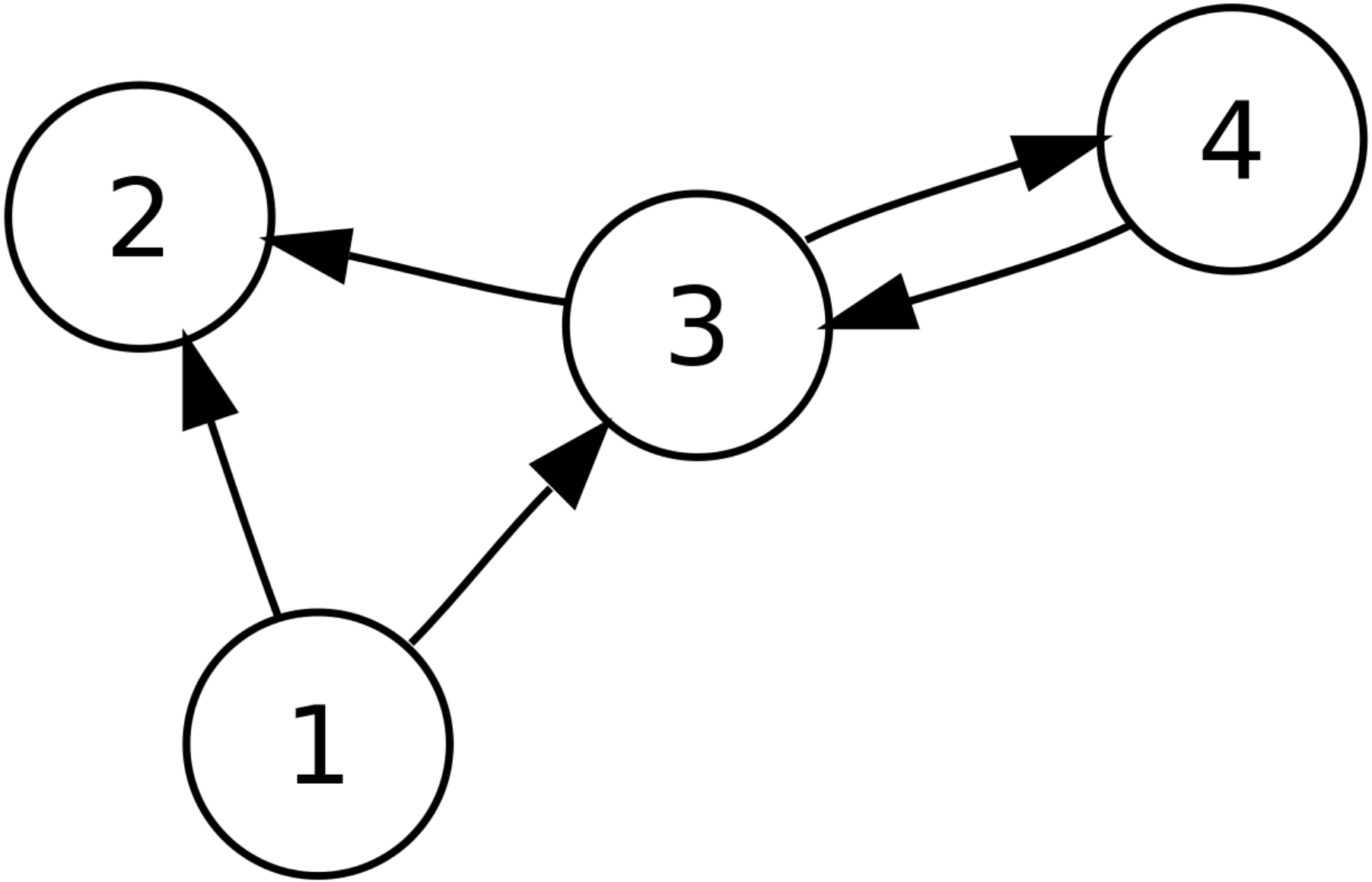
Traduction

```
modify(Tab,Pt,1) = TabP -- <Tab,Pt,+> => <TabP,Pt>
```

Comment définir modify?

injection python?

Interface (visualisation par graphe)



outils de visualisation

Affichage

`NetworkX + matplotlib`

Interactivité

`NetworkX + wxPython`